

REAL-TIME SIMULATION OF MOMENTUM-CONSERVING VORTEX DYNAMICS VIA AFFINE VELOCITY APPROXIMATION*

ALFIE KUNZ[†]

PROJECT SUPERVISOR: DR. RADU CIMPEANU[†]

Abstract. We detail the derivation, understanding, and implementation of the hybrid Eulerian-Lagrangian Particle-In-Cell (PIC) model, and its successor Affine-PIC, for efficiently resolving vortical motion in a high Reynolds number flow. By constantly interpolating between particle-based velocities, in which advection is performed, and grid-based velocities, where projection is enforced by an optimised Conjugate Gradient Method, we introduce a loss of degrees of freedom in the system which manifests as dissipation. APIC subsequently augments particles with affine velocity gradient matrices, conserving angular momentum across the transfers, whilst still allowing for shearing of fluid elements. Comparison to the analytical ‘Taylor-Green’ vortex yields a Reynolds number $Re = 5420$ - a “first of its kind” result in the field of real-time CFD.

Key words. particle-in-cell (PIC) method, affine PIC, computational vortex dynamics

1. Introduction. Effectively simulating the self-advective nature of fluid, despite current computational processing power allowing for sophisticated and diverse numerical methods, remains a central challenge in modern computational fluid dynamics. The emergence of vortices in incompressible fluids comes directly from their numerical description through the Navier-Stokes equations, and whose resultant forces are essential for describing the simplest of fluid dynamics. For example, resolving the D’Alembert paradox for free cylinders in a stream [15] allows us to study the oscillatory forces necessary for many modern wind turbines [1], or vortex shedding to produce aerofoil lift (where numerical inaccuracies can cause mismatched behaviours for large-scale planes) [26]. As such, the criticality of modelling such behaviours, in a way that conserves the energy and momentum of the system, cannot be overstated.

In particular, this investigation places an emphasis on the large scale applicability of these models, as to justify the application to, for example, the aviation industry. We thus search for a versatile, *highly efficient* method, that is able to produce and sustain turbulence that agrees with analytical solutions. After providing the core mathematical and intuitive background of solving a general fluid on the grid, we will derive the Particle In Cell (PIC) method in Section 2, which is able to simulate roughly 30,000 particles in real time (120 simulation steps per second) on a single-core, laptop processor. Whilst these models are built around Euler’s Equations [8], which assumes an *inviscid* flow, natural viscosity can arise from the numerical dissipation from particle-to-grid transfers, which breaks our assumptions and limits turbulent motion. Section 3 is therefore dedicated to the discussion and derivation of several techniques, most notably Affine PIC (APIC) that mathematically eliminate certain modes of dissipation. Subsequent discussion, comparison to analytical, inviscid solutions, and reflection, will follow in Sections 4-5.

Alongside its computational efficiency, PIC scales competently: as the particle count increases, the interpolation errors for divergence and angular momentum exhibit asymptotically linear time-dependence. Combined with the external performance benefits from writing these methods as a GPU compute shader, we can potentially simulate millions of particles in real-time, for large-scale applications. APIC in par-

*Submitted to SIURO April, 2026.

[†]MATHEMATICS INSTITUTE, UNIVERSITY OF WARWICK, CV4 7AL, UNITED KINGDOM (ALFIE.KUNZ@WARWICK.AC.UK, RADU.CIMPEANU@WARWICK.AC.UK).

ticular has made its mark across industries, such as Walt Disney Animation Studios' implementation to simulate highly-detailed crashing waves for 2016's Moana [2].

1.1. Eulerian, Lagrangian, and Hybrid Frameworks. Lagrangian fluids are represented by infinitesimal point particles, whilst Eulerian schemes are discretised into a fixed grid of small cells. All respective 'units' exert forces on one another as described by the Navier-Stokes equations. Naturally, each model has its own distinct computational advantages. Lagrangian models are well-suited for *advection*, as we can move particles via Semi-Euler integration or 'Runge-Kutta (2nd Order)' using known velocity data. In fact, Eulerian models typically combat this with an approach called *Semi-Lagrangian Advection*, in which we solve for the initial location of some 'virtual' particle that *would have travelled* to a grid cell of interest [31]. Particle representations further excel in modelling *inviscid* flows, and for effortless computation on the shape of a fluid package - a complicated task for grid-based systems (where we only have access to the *relative* pressures of each cell). On the other hand, 'Finite Difference' Method (FDM) calculations become non-trivial for Lagrangian schemes, giving them a distinct weakness for *projection*. We typically solve this by applying a Gaussian kernel to each particle, creating a low-accuracy approximation of a density field - this forms a pseudo-pressure gradient on particles, which is used to correct the velocity field as in Section 2.2.1. Contrarily, Eulerian models, with fixed cells that mould to FDM calculations effortlessly, excel in projection and boundary handling.

This motivates the discussion of *hybrid* fluid models, in which we constantly interpolate between particle-based and grid-based approaches, to model the physical aspects in which each respective method thrives. The classical Particle-In-Cell (PIC) method, dating back to 1955 [14] is a popular choice, as transfer schemes dampen noise very effectively. This provides stability, at the cost of exponential energy losses. This paper covers two classes of solutions: FLIP [5], which conserves energy during transfers, at the result of numerical instability, and APIC [22], which conserves the natural angular momentum of particles during transfers, whilst remaining stable.

2. PIC: Particle-In-Cell.

2.1. Discrete Operations, and the Staggered Grid. We define our Eulerian fluid as a uniform grid of square cells, with sufficiently small length s , as to be compatible with FDM calculations. As such, fluid boundaries are represented via the 'Cut Cell' method of [8], rather than the complex boundaries attainable using Finite Volume methods. Whilst being efficient, this choice is appropriate as we are focussed on the inertial terms present in the Navier Stokes' equations, rather than boundary-dominated terms.

Assuming a smooth velocity field $\underline{u}(x, y)$, and scalar-valued field $p(x, y)$ with values known only at discrete grid points, we discretise the spatial differential operators,

(2.1a)

$$(\nabla p)(x, y) = \left(\frac{\partial p}{\partial x}, \frac{\partial p}{\partial y} \right) = \frac{p(x + \frac{s}{2}, y) - p(x - \frac{s}{2}, y)}{s}, \frac{p(x, y + \frac{s}{2}) - p(x, y - \frac{s}{2})}{s},$$

$$(\Delta p)(x, y) = \frac{\partial^2 p}{\partial x^2} + \frac{\partial^2 p}{\partial y^2}$$

$$= \frac{p(x + s, y) + p(x - s, y) + p(x, y + s) + p(x, y - s) - 4p(x, y)}{s^2}.$$

Applied to a *finite, regular grid*, these allow us to describe Euler's equations for an Eulerian fluid. Since the forthcoming Poisson equation (2.3) equates $\nabla \cdot \underline{u}$ and Δp , it seems natural to *stagger* the velocities for each grid cell via $u_x(x_{i+\frac{1}{2}}, y_j) \mapsto u_x(x_i, y_j)$ and $u_y(x_i, y_{j+\frac{1}{2}}) \mapsto u_y(x_i, y_j)$, and thus store the *pressure* of each cell at each cell's centre. This particular grid is called 'Marker-And-Cell' (MAC) [7], with representation as depicted in Figure 1, and comes alongside the following notation.

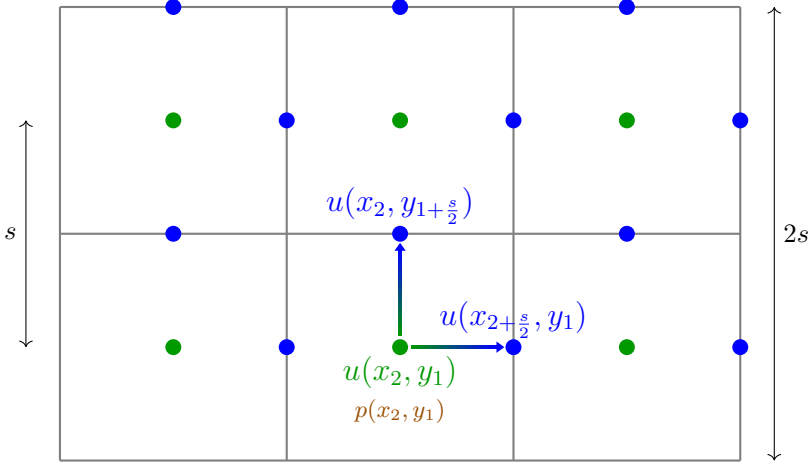


Figure 1: A small-scale example of a Staggered MAC Velocity Grid.

Notation 2.1.

- We denote $\mathbf{i}(a, b) := \{sa, s(b + \frac{1}{2})\}$, $\mathbf{j}(a, b) := \{s(a + \frac{1}{2}), sb\}$ as the **positions** of the velocity grid cell, at index (a, b) .
- We denote $\mathbf{u}(a, b) := \{u_x(\mathbf{i}(a, b)), u_y(\mathbf{j}(a, b))\}$ as the **velocity** of the grid cell at (a, b) .
- We denote $\mathbf{p}(a, b) := p(s(a + \frac{1}{2}), s(b + \frac{1}{2}))$ as the **pressure** of the grid cell at (a, b) .

This representation of our fluid is now well-handled for enforcing a divergence-free flow.

2.2. Solving for Divergence-Free Velocity.

2.2.1. Potentials and Pseudo-Pressures. The primary goal of our solver will be to enforce mass conservation $\nabla \cdot \underline{u} = 0$; first, we discover a powerful tool for expressing our velocity field: *Helmholtz Decomposition*.

THEOREM 2.2 (Helmholtz Decomposition in $\mathbb{R}^2$). *For some $V \subseteq \mathbb{R}^2$, any vector field $u \in C^1(V, \mathbb{R}^2)$ can be written in the form:*

$$u(x, y) = \mathcal{C}(x, y) + \mathcal{D}(x, y),$$

for some $\mathcal{C}, \mathcal{D} \in C^1(V, \mathbb{R}^2)$, where $\nabla \times \mathcal{C} = 0$ and $\nabla \cdot \mathcal{D} = 0$ [4].

To prove this theorem, we follow C. Baird [4] in applying Stokes' Theorem to a standard Dirac-Delta's identity property of convolution.

Remark 2.3. It is for this reason that we call solving for a divergence-free flow *projection* - we are essentially **projecting** our vector field $\underline{u}$ onto the divergence-free basis vector.

Using Theorem 2.2, and the fact that, for each differentially-continuous curl-free vector field on a simply connected set, there exists a scalar potential whose gradient is that vector field [30], we consider the equation

$$(2.2) \quad \underline{u}_{new}(x, y) = \underline{u}_{df}(x, y) = \underline{u}(x, y) - \underline{u}_{cf}(x, y) = \underline{u}(x, y) - \nabla p.$$

Taking the divergence yields

$$(2.3) \quad \nabla \cdot \underline{u} = \Delta p,$$

motivating a plan to project our fluid: use its current divergence to calculate the Laplace of some scalar-valued p , then subtract its gradient from $\underline{u}$ to generate a divergent-free flow [13]. In particular, we consider p to be the pressure gradient found in the Navier-Stokes equations. Indeed, a build-up of pressure in some fluid element is directly correlated with a net flux of fluid into it, and thus a violation of mass conservation. However, as we are using $\underline{u}$ to define an ‘effective pressure’, rather than calculating p explicitly, we call p a *pseudo-pressure gradient*.

We finally consider the boundary condition of defining the pressure of a cell that contains no particles, be it either a solid or air cell. Modelling air as a vacuum enforces $p_{\text{air}} = 0$; a more sensible (and computationally efficient) construction is to restrict the function p to fluid cells *only*, and access air cell pressures, for use in formula (2.1a), on-the-fly. This has the bonus effect of also restricting $\underline{u}_{df}$, meaning we never have to define the velocities between neighbouring air cells. To tackle solid cells, we make use of the *Neumann boundary condition* [8] $\underline{u}(x, y) \cdot \underline{n} = 0$, for cells (x, y) that neighbour a solid, with normal $\underline{n}$. Via the impermeability boundary condition, this enforces $p_{\text{solid}} = \underline{u}(x, y)$.

2.2.2. Forming a Linear Equation. Applying the discrete operations (2.1b) (along with the discrete divergence operator of a vector field) to equation (2.3), we are able to define the divergence of some some cell with index (a, b) [13], giving

$$(2.4) \quad \frac{d(a, b)}{s} := \frac{\underline{u}_x(a, b) - \underline{u}_x(a - 1, b) + \underline{u}_y(a, b) - \underline{u}_y(a, b - 1)}{s} \\ = \frac{\mathbf{p}(a + 1, b) + \mathbf{p}(a - 1, b) + \mathbf{p}(a, b + 1) + \mathbf{p}(a, b - 1) - 4\mathbf{p}(a, b)}{s^2}.$$

Denoting F as the set of total cell indices (a, b) at time t , with cardinality n_t , and C as the set of all cell indices (a, b) , we can form the bijective map $f : \llbracket n_t \rrbracket \rightarrow F$, where $\llbracket n_t \rrbracket := \{1, 2, \dots, |n_t|\}$. In the following text, we often abuse notation and write $\mathbf{p}(f(i)) \rightsquigarrow \mathbf{p}(i)$ and $d(f(i)) \rightsquigarrow d(i)$. Defining $N(i) : \llbracket n_t \rrbracket \rightarrow \{0\} \cup \llbracket 4 \rrbracket$ as the number of *non-solid direct neighbours* the cell with index $f(i)$ has, we can apply formula (2.4) to every fluid cell, to produce a linear equation representation of formula (2.3):

$$(2.5) \quad -\mathbf{A}\underline{\mathbf{p}} := -\mathbf{A} \begin{pmatrix} \mathbf{p}(f(1)) \\ \mathbf{p}(f(2)) \\ \vdots \\ \mathbf{p}(f(n_t)) \end{pmatrix} = \begin{pmatrix} d(f(1)) \\ d(f(2)) \\ \vdots \\ d(f(n_t)) \end{pmatrix} =: s\underline{d},$$

144 where $\mathbf{A} \in \mathbb{R}^{n_t, n_t}$, referred to as the *Coefficient Matrix*, is defined via

$$145 \quad (2.6) \quad A(i, j) = \begin{cases} N(i) & \text{if } i = j \\ -1 & \text{if } f(i) \text{ is a neighbour of } f(j) \\ 0 & \text{else} \end{cases}$$

146 $\mathbf{A}$ therefore represents the *negative discrete Laplace Operator* $-\Delta$, which will become
 147 increasingly sparse as n_t grows. We will take for granted the fact that $\mathbf{A}$, as defined
 148 in (2.6), is *symmetric*, sparse, and *positive-definite* when restricted to multiplication
 149 by non-constant vectors. Note that this restriction is valid as equation (2.3) reveals
 150 that such a case would result in $\underline{u}$ already being incompressible.

151 **LEMMA 2.4.** *The matrix $\mathbf{A}$, as defined in (2.6), is symmetric, and positive-definite*
 152 *when restricted to multiplication by non-constant vectors.*

Proof. Symmetry follows immediately from the definition of $\mathbf{A}$ and a ‘neighbour’. Recall that $\mathbf{M} \in \mathbb{R}^{n, n}$ is positive-definite if $\underline{p} \cdot \mathbf{M} \underline{p} > 0$, $\forall \underline{p} \in \mathbb{R}^n \setminus \{0\}$. Applying this to $\mathbf{A}$ gives:

$$\underline{p} \cdot \mathbf{A} \underline{p} = \sum_{i=1}^{n_t} p_i \left(N(i) p_i - \sum_{\text{Nbs}(i)} p_j \right) = \sum_{i=1}^{n_t} \left(N(i) p_i^2 - \sum_{\text{Nbs}(i)} p_i p_j \right),$$

where $\text{Nbs}(i) := \{j \in \llbracket n_t \rrbracket \mid f(i) \text{ is a neighbour of } f(j)\}$. As $|\text{Nbs}(i)| = N(i)$, for each neighbour pair p_a and p_b we can bijectively assign a p_a^2 from $N(a)p_a^2 = \underbrace{p_a^2 + p_a^2 + \dots}_{N(a)}$,

and a p_b^2 from $N(b)p_b^2$, to form

$$p_a^2 - \overbrace{p_a p_b}^{i=a \text{ loop}} - \overbrace{p_b p_a}^{i=2 \text{ loop}} + p_b^2 = (p_a - p_b)^2.$$

From this, we can remove the neighbour pair (a, b) from the summation loops to get

$$\underline{p} \cdot \mathbf{A} \underline{p} = \sum_{i=1}^{n_t} \left(N(i) p_i^2 - \sum_{\text{Nbs}(i)} p_i p_j \right) + (p_a - p_b)^2.$$

153 Repeating for all neighbour pairs (i, j) (decrementing $N(a)$, $N(b)$ for each pair) sets
 154 $N(i) = 0 \forall i$ via our bijective assignment. Under non-constant vectors $\underline{p}$, we get the
 155 strictly positive quantity,

$$156 \quad (2.7) \quad \underline{p} \cdot \mathbf{A} \underline{p} = \sum_{\text{neighbours } (i, j)} (p_i - p_j)^2. \quad \square$$

157 The resulting symmetric, positive-definite linear system is solved through the
 158 Conjugate Gradient Method [32].

159 **2.2.3. The Conjugate Gradient (CG) Method.** We consider the solution
 160 to the linear equation,

$$161 \quad (2.8) \quad \mathbf{A} \underline{p} = -s^2 \underline{d} =: \underline{b},$$

where $\mathbf{A}$ is a matrix with construction as in formula (2.6), and $\underline{d}$ as in formula (2.4). As $\mathbf{A}$ is symmetric and positive-definite, the inner product,

$$\langle \underline{u}, \underline{v} \rangle_A := \underline{u} \cdot \mathbf{A} \underline{v},$$

is well-defined: two vectors are called conjugate (rather, orthogonal through $\mathbf{A}$) is this is equal to 0 [3].

As the set of mutually conjugate vectors $X := \{\underline{x}_1, \dots, \underline{x}_{n_t}\}$ forms a basis for $\mathbb{R}^{n_t}$, calculating the related coefficients $\alpha_i \in \mathbb{R}$ for $\underline{b}$ would allow us to solve equation (2.8) [8, 32]. Pre-multiplying the basis expansion of equation (2.8) by some $\underline{x}_k$ and applying the definition of orthogonality, we thus define these coefficients as,

$$(2.9) \quad \alpha_k = \frac{\underline{x}_k \cdot \underline{b}}{\langle \underline{x}_k, \underline{x}_k \rangle_A}.$$

This process is intractable for large n_t (the number of fluid cells) - rather, it is more elegant to build each $\underline{x}_i$ inductively, generating a *reasonable* approximation for $\underline{b}$ when stopping after $m < n_t$ iterations.

To choose each appropriate $\underline{x}_k$, we exploit the fact that $\mathbf{A}$ is the Hessian of the *quadratic form* [32], given by

$$(2.10) \quad f(\underline{\mathbf{p}}) = \frac{1}{2} \underline{\mathbf{p}} \cdot \mathbf{A} \underline{\mathbf{p}} - \underline{\mathbf{p}} \cdot \underline{b},$$

and thus aim to descend its contours via *gradient descent*, towards the minimiser $f(\underline{\mathbf{p}}_\infty)$ (where $\underline{\mathbf{p}} = \underline{\mathbf{p}}_\infty$ satisfies $\nabla f(\underline{\mathbf{p}}_\infty) = \mathbf{A} \underline{\mathbf{p}}_\infty - \underline{b} = \underline{0}$).

This requires the construction of a residue $\underline{r}_0 = -\nabla f(\underline{\mathbf{p}}_0)$, given the initial guess $\underline{\mathbf{p}}_0$: we write (2.9) as

$$\alpha_k \rightarrow \alpha(\underline{r}_k) = \frac{\underline{x}_k \cdot \underline{r}_k}{\langle \underline{x}_k, \underline{x}_k \rangle_A}.$$

To ensure that $\underline{x}_k$ is mutually conjugate to all other $\{\underline{x}_1, \dots, \underline{x}_{k-1}\}$, we construct it by applying the *Gram-Schmidt Orthonormalization* process [3, 8], to remove any vector projections in all previous conjugate vectors. This gives the iterative solution $\underline{\mathbf{p}}_{k+1} = \underline{\mathbf{p}}_k + \alpha_k \underline{x}_k$, with error $\underline{r}_{k+1} = \underline{b} - \mathbf{A} \underline{\mathbf{p}}_{k+1}$. JR Shewchuk [32] outlines a full proof that $\lim_{k \rightarrow n_t} \underline{\mathbf{p}}_k = \underline{\mathbf{p}}_\infty$, and thus we terminate our algorithm as $\|\underline{r}\|_\infty := \max\{r_1, \dots, r_{n_t}\}$ passes below some threshold. We use the l_∞ norm over l_2 as it corrects the *worst-case* areas of divergent flow [8]; otherwise, we might have realistic fluid in an averaged sense, apart from some small areas that (locally) behave poorly.

We reuse the pressure values for each cell of the previous timestep as the solver's initial guess [8], which empirically reduces the number of iterations by approximately 20%, relative to $\underline{\mathbf{p}}_0^t = \underline{0}$.

2.2.4. Preconditioning. It may be that the CG algorithm, performed on the linear system $\mathbf{M}^{-1} \mathbf{A} \underline{\mathbf{p}} = \mathbf{M}^{-1} \underline{b}$ (where $\mathbf{M}$ is some symmetric, positive-definite matrix in $\mathbb{R}^{n_t, n_t}$), converges much faster than the standard formula (2.8). Hence, it is often a good use of computation time to first construct a *pre-conditioner* $\mathbf{M}$, as a viable approximator of $\mathbf{A}$, for use in solving the *Preconditioned* Conjugate Gradient (PCG) Method [32].

The CG algorithm converges well if $\mathbf{A}$'s eigenvalues are packed together, meaning the quadratic form f in (2.10) is locally steep about $\underline{\mathbf{p}}_\infty$ [8]. This emerges for particularly laminar, viscous flow (where $\underline{\mathbf{p}}$ varies slowly between neighbouring cells). Hence, $\mathbf{M}$ is a good choice for our pre-conditioner if it is sparse and easy to compute from $\mathbf{A}$, yet is such that $\mathbf{M}^{-1} \mathbf{A}$'s eigenvalues are all close to unity. We follow R. Bridson's recommendation [8] of '(Modified) Incomplete Cholesky Factorisation with (Drop-Tolerance) Thresholding, Level-Zero (MICT-0)'.

Let the symmetric $\mathbf{M}$ be factorised into the lower & upper triangular matrix product $\mathbf{L}\mathbf{L}^T$, and consider the linear equation,

$$(\mathbf{L}^{-1}\mathbf{A}[\mathbf{L}^{-1}]^T)(\mathbf{L}^T\mathbf{p}) = \mathbf{L}^{-1}\mathbf{b},$$

which has identical solutions to that of (2.8). As $(\mathbf{L}^{-1}\mathbf{A}[\mathbf{L}^{-1}]^T)$ is symmetric and positive-definite, we can apply the CG Method as before, but now, after constructing $\mathbf{r}_k$, we *precondition* to get $\mathbf{z}_k$, the vector which solves,

$$(2.11) \quad \mathbf{M}\mathbf{z}_k = \mathbf{r}_k,$$

and use $\mathbf{z}_k$, rather than $\mathbf{r}_k$, for calculating $\mathbf{x}_k$ in Gram-Schmidt. Taking advantage of $\mathbf{M}$'s new decomposition, we can easily solve these sub-systems by breaking into a set of sub-equations and applying basic forward and backward substitutions [8].

Finally, we consider the MICT-0 factorisation of $\mathbf{M} = \mathbf{L}\mathbf{L}^T$ by approximating $\mathbf{L}$ via some lower triangular matrix $\mathbf{K}$, which retains $\mathbf{A}$'s sparsity, but still well-approximates it, with $A \approx M := \mathbf{K}\mathbf{K}^T$ positive-definite. The ‘thresholding’ technique in this factorisation defines the critical value τ under which $\mathbf{A}$'s entries are deemed to have a negligible effect on the description of the fluid, vs the preservation of sparsity [25]. We compute $\mathbf{K}$ via the following algorithm [18, 25]:

$$\mathbf{K}_{ij} = \begin{cases} \sqrt{w\mathbf{K}_{ii}} & i = j \\ w & \text{if } j < i, |w| \geq \tau \\ 0 & \text{if } j < i, |w| < \tau \\ 0 & j > i \end{cases}; \quad \text{where } w = \frac{\mathbf{A}_{ij} - \sum_{k=1}^{j-1} \mathbf{K}_{ik}\mathbf{K}_{jk}}{\mathbf{K}_{jj}}.$$

The MICT-0 algorithm, alongside the PCG Method, is implemented as detailed in the [accompanying repository](#).

2.3. Bilinear Interpolation. To transfer velocities between particles and the grid, we use bilinear interpolation’ [13], which can be represented via the respective linear operators $\mathbf{u}_{\text{grid}} = \text{BiLerpPG}_{\mathbf{x}_p}(\mathbf{u}_p)$ (for particles to grid) and $\mathbf{u}_p = \text{BiLerpGP}_{\mathbf{x}_p}(\mathbf{u}_{\text{grid}})$, where $\mathbf{x}_p$ denotes particle positions (and $\mathbf{x}_{\text{grid}}$ are pre-determined via the MAC format). Considering BiLerpPG, for example, we first assign eight *weights* $\{w_p^1, w_p^2, \dots, w_p^8 : w_p^i \in [0, 1]\}$ for each particle p , each holding the relative distances from $\mathbf{x}_p$ to some specific point on the grid. In our case, $\{w^1, \dots, w^4\}$ represents the relative distances to p 's four closest $\mathbf{u}_x$ locations on the grid, and $\{w^5, \dots, w^8\}$ represents the relative distances to the four closest $\mathbf{u}_y$ locations. We can see this visually in Figure 2.

where Δx and Δy represent the horizontal and vertical distance from the particle to the bottom-left MAC velocity component of question, and s again denotes cell width [8]. Taking a weighted average of p 's weights (to conserve linear momentum) yields the interpolation equation:

$$(2.12) \quad \mathbf{u}_x(a, b) = \frac{\sum_{1 \leq i \leq 4} \sum_p w_p^i \mathbf{u}_p}{\sum_{1 \leq i \leq 4} m_i^x}, \quad \mathbf{u}_y(a, b) = \frac{\sum_{5 \leq i \leq 8} \sum_p w_p^i \mathbf{u}_p}{\sum_{5 \leq i \leq 8} m_i^x},$$

where we define $m_i^x = \sum_p w_p^i$ and $m_i^y = \sum_p w_p^i$ as the ‘mass effect’ of p on the grid node i . In the discussion to follow, equations (2.12) are combined to form the single interpolation equation:

$$(2.13) \quad \mathbf{u}_{\text{grid}} = \frac{\sum_i \sum_p w_p^i \mathbf{u}_p}{\sum_i (m_i := \sum_p w_p^i)}.$$

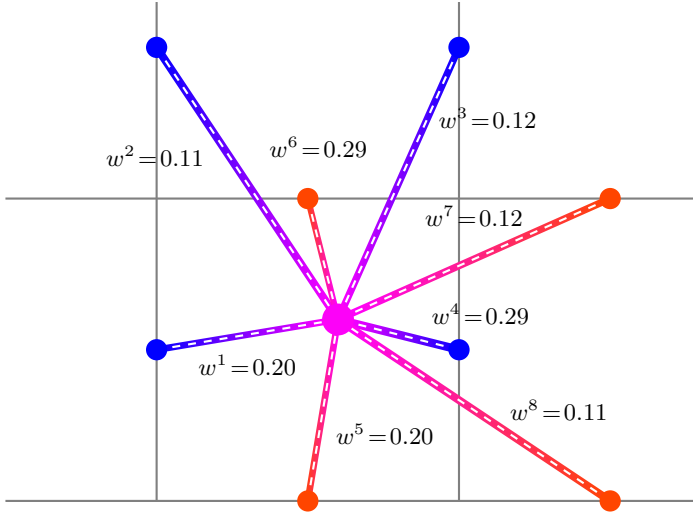


Figure 2: Construction of weights w_p^i for Bilinear Interpolation (Particle-to-Grid) on a MAC Grid. The blue nodes represent the MAC x-velocity locations $\{w^1, \dots, w^4\}$, and the orange nodes represent the y-velocity locations $\{w^5, \dots, w^8\}$, with the pink particle at the centre representing the location of $\underline{x}_p$.

A similar process can also be formed for BiLerpGP. Also note that, via our Dirichlet boundary conditions, we must enforce $w_p^{solid} = 0$: no particle velocity can be interpolated to a solid cell.

3. Vortex Analysis. We begin with a brief reflection on what we have covered thus far, in deriving PIC - specifically, we consider the emergence of vortex behaviour, and the angular momentum properties which drive it. This will motivate the need to upgrade our model with new energy modes, or velocity gradient information, which allow for better deformation of fluid elements. Finally, we will quantify the total angular momentum of our fluid, in order to compare methods.

3.1. Problems with PIC. With our Particle-In-Cell method now in place, one discovers that fluid appears overly smooth and gelatinous, despite our specifically considering inviscid, high Reynolds number flows, in which we can drop viscosity term $\nu \nabla^2 \underline{u}$ in the Navier-Stokes equations. Put simply, *numerical dissipation* is the root cause of this issue: a fatal flaw in our hybrid models is that we are constantly *smoothing* our particle and grid velocities when interpolating to the other respective scheme [29]. This averaging nature results in a large system energy loss, presented in the form of high viscosity, and thus a lack of vortices [29].

Written another way, any artificial noise added to particle velocity is filtered out via the respective transfer schemes. The physical interpretation of this lies in the system's *degrees of freedom* (DoF) - we allocate 4 particles to each grid cell (with 8 total DoF), the information of which is then condensed into the grid's x and y velocity components - 2 DoF. I.e: the bilinear operator has a kernel of dimension 4 [11], producing the smoothing effect we are wishing to rectify.

We consider two approaches in remedying this uninspiring behaviour: *FLIP* (*Fluid-Implicit Particle*), whose aim in reducing numerical dissipation is set by con-

serving energy in particle-grid transfers, and *APIC* (*Affine Particle-In-Cell*), which attempts to quantify and represent angular momentum losses of particles in the grid representation.

3.2. FLIP: Fluid-Implicit Particle. FLIP is the simpler of the two methods, and operates as such: we interpolate from the particles to the grid as normal, then make a *full copy* of the grid velocities before any work is undergone. After modifying the grid velocities due to projection, we instead interpolate the *change in this grid velocity update* back to the particles, adding this to their current velocities. This effectively removes a smoothing step, and hence a large source of our numerical dissipation [5]. One can show that this scheme conserves the internal energy and total momentum across the transfers, and that the loss of kinetic energy is only non-negligible when the velocity distributions of particles is poorly represented by a bilinear interpolation function. This is the case only for strongly accelerated flow, where our solver struggles to interpret $\nabla \underline{u}$ accurately [5, 6]. As our bilinear interpolation operator has a non-trivial kernel, it is possible for some artificial noise added to the particles in some cell to not be transferred to the grid; by our FLIP scheme, no correction / dampening term will be added back to the particles. Hence, FLIP is an inherently *unstable* scheme [11]. We can see this numerical instability in Figure 3.

Linearly interpolating between the PIC and FLIP models [8] allows us to fine-tune the system's viscosity we define the *blending factor* $\alpha \in [0, 1]$, and update particle velocities via a superposition of interpolations:

$$\underline{u}_p = \underbrace{\alpha \cdot \text{BiLerpGP}_{x_p}(\underline{u}_{grid}^{new})}_{\text{PIC}} + (1 - \alpha) \cdot \underbrace{\left[\underline{u}_p^{old} + \text{BiLerpGP}_{x_p}(\underline{u}_{grid}^{new} - \underline{u}_{grid}^{old}) \right]}_{\text{FLIP}}.$$

One can relate α to the *kinematic viscosity* of the fluid, ν , via $\alpha = 6\nu\Delta t/\Delta x^2$ [8], from which we can follow the well-supported value of $\alpha = 0.05$ - a good approximator of the natural viscosity of water [8]. This produces the most visually pleasing and intuitively accurate simulation we have seen thus far in Figure 4, and finally allows for perceptible vortex motion in a stable environment.

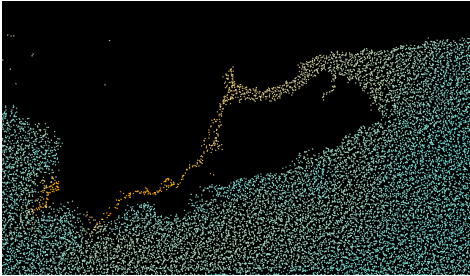


Figure 3: A Low-Resolution, FLIP Simulation - note the lack of smoothing on the left side, where particles of vastly different velocities are located very close together.

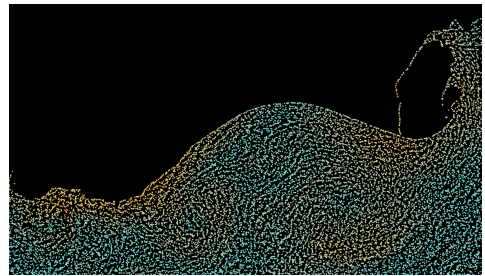


Figure 4: Mimicked results for PIC/FLIP ($\alpha = 0.05$) - this greatly improves upon PIC, allowing for shearing and rotational deformation whilst mostly inheriting PIC's stability.

We see that, whilst FLIP preserves energy across time-steps, and thus rotational motion, necessitating *any* amount of PIC ($\alpha > 0$) in order to retain stability ensures that angular momentum is not conserved across multiple simulation steps.

3.3. APIC: Affine Particle-In-Cell.

3.3.1. Conservation of Angular Momentum. From the previous discussion, it is rather clear how we can reduce the numerical dissipation of our transfer operators, without resorting to unstable schemes - add more DoF to our grid representation [11] (through the BiLerpGP operator, in particular). Specifically, our studies will focus on augmenting each cell with a linear operator (representing the angular momentum loss in the particle representation) in a way that does not affect our CG Algorithm.

First, however, we aim to derive the amount of momentum we lose via our particle-grid transfers. This is specifically restricted to the *angular* kind - our bilinear interpolation operators (2.13) by construction conserve linear momentum [22]. At some time t , let our MAC grid have divergence-free (as in, post projection) velocity components $\underline{u}_g$ at fixed positions $\underline{\chi}_g$, and thus weights w_p^g from some particle p . Formally, these are given by the collection of grid nodes $g \in G$, with velocities

$$\left\{ \begin{pmatrix} \mathbf{u}(a, b)_x \\ 0 \end{pmatrix}, \begin{pmatrix} 0 \\ \mathbf{u}(a, b)_y \end{pmatrix} \right\}, \quad \text{at positions } \{i(a, b), j(a, b)\}.$$

Then, we can form an equation for the angular momentum of each of our representations,

$$(3.1) \quad L_P = \sum_p \underline{x}_p \times \underline{u}_p, \quad L_G = \sum_g \underline{\chi}_g \times m_g \underline{u}_g = \sum_g \sum_p w_p^g (\underline{\chi}_g \times \underline{u}_p),$$

where $\sum_p$ is over all particles within the interpolation range of the grid cell g , and $m_g \underline{u}_g = \sum_i \sum_p w_p^g \underline{u}_p$ is a restated form of equation (2.13) [22]. Also note that, as $\underline{x}, \underline{u}$ live in $\mathbb{R}^2$, L_P and L_G are scalar values. The angular momentum loss for some particle p in the BiLerpGP operator, therefore, compared to its representation on the grid, comes directly from ‘pivoting’ it around each grid point g (with position $\underline{\chi}_g$) it influences, and crossing this with that grid point’s momentum:

$$(3.2) \quad L_p^{\text{loss}} = \sum_g w_p^g (\underline{\chi}_g - \underline{x}_p^{\text{old}}) \times \underline{u}_g,$$

where $\underline{x}_p^{\text{old}}$ is p ’s position *before* advection (as opposed to $\underline{x}_p$, which is defined afterwards). Trivially, our interpolation operators conserve momentum if the particles are placed directly on top of the grid points, and errors accumulate linearly as the particles begin to distribute. We will thus attempt to enrich our particle velocities with these L_p^{loss} ’s, in the attempt to induce them with more DoF [35]. In other words, we take the loss in angular momentum in BiLerpGP, and feed this into the interpolation velocities for the next simulation step’s BiLerpPG, thus locally conserving the quantity for *both* operators [22].

DEFINITION 3.1. We call some function $f : X \rightarrow Y$ locally... [11]

- **constant** if, $\forall x \in X$, $\exists$ an open neighbourhood $U \subset X$, containing x , such that f restricted to U is constant: $f(u) = f(x)$, $\forall u \in U$.
- **affine** if, $\forall x \in X$, $\exists$ an open neighbourhood $U \subset X$, containing x , such that f restricted to U is linear: $f(u) = f(x) + \mathbf{L}(u - x)$, $\forall u \in U$, and where $\mathbf{L} \in \mathcal{L}(X, Y)$.
- **rigid** if it is affine, with the extra condition that L is anti-symmetric ($\mathbf{L}^T = -\mathbf{L}$).

In these PIC transfer operations, we essentially treat each particle as *locally constant* (where this ‘local’ behaviour is apt on the scale of the size of a grid cell). We first introduce the Rigid Particle-In-Cell improvement, which treats each particle as *locally rigid*.

3.3.2. Rigid Approximations. From Definition 3.1, we can think of a locally rigid function as one which *preserves distances* in some open neighbourhood $U \ni x$: $|f(u) - f(x)| = |u - x|$, $\forall u \in U$. This term *rigid* comes from rigid-body physics, in which we assume that some small body (or fluid element) cannot deform.

Attaching a locally rigid velocity field $u \in C^2(\mathbb{R}^2, \mathbb{R}^2)$ to each particle p at position $\underline{x}_p$ gives $u(\underline{x}) = u(\underline{x}_p) + \mathbf{R}_p(\underline{x} - \underline{x}_p)$, where $\mathbf{R}_p \in \mathcal{L}(\mathbb{R}^2, \mathbb{R}^2)$ is anti-symmetric, and represents a local approximation of ∇u [35]. Relating this to the physics of combining linear and angular momentum, we can relate $\mathbf{R}_p$ to an angular velocity operator, and thus write,

$$(3.3) \quad \omega_p^* = \mathbf{R}_p, \quad \text{where} \quad \omega_p^* a = \underline{\omega}_p \times \underline{a} = \omega \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} \alpha_x \\ \alpha_y \end{pmatrix}.$$

Generally, we relate angular velocity to angular momentum via the equation $\underline{L}_p = \mathbf{I}_p \underline{\omega}_p$, where

$$(3.4) \quad (\mathbf{I}_p)_{ij} := \sum_k m_k (r_k^2 \delta_{ij} - (x_k)_i (x_k)_j) dV \in \mathbb{R}^{3,3},$$

is the inertia tensor (representing a body’s natural resistance to rotation) [17, Equations 5-6, 5-8] of the rigid body χ that p contributes to during interpolation - that being the family of MAC velocity component points at locations $\underline{\chi}_g$, with weights $\{w_p^1, \dots, w_p^8\}$. As ω and L are scalar values (representing a vector perpendicular to $\underline{u}_p$), we can write equation (3.4) as

$$(3.5) \quad \underbrace{I_p := I_{zz}}_{\cong (x_{p \rightarrow g}^2 + y_{p \rightarrow g}^2)} = \sum_g w_p^g \underbrace{\|\underline{\chi}_g - \underline{x}_p\|^2}_{\Delta x^{gp}} = \sum_g w_p^g [(\Delta x_x^{gp})^2 + (\Delta x_y^{gp})^2].$$

Hence, following C. Jiang [22] by combining equations (3.2) and (3.5) gives,

$$(3.6) \quad \omega_p = \frac{L_p^{\text{loss}}}{I_p} = \frac{\sum_g w_p^g [(\underline{\chi}_g - \underline{x}_p^{\text{old}}) \times \underline{u}_g]}{\sum_g w_p^g \|\underline{\chi}_g - \underline{x}_p\|^2},$$

and so the $\sum_p w_p^i \underline{u}_p$ term in equation (2.13) becomes

$$(3.7) \quad \sum_p w_p^i \underline{u}_p^{\text{rigid}}, \quad \text{where} \quad \underline{u}_p^{\text{rigid}} = \underline{u}_p + \omega_p^* (\underline{\chi}_i - \underline{x}_p).$$

Applying this to our BiLerpPG operator conserves both linear and angular momentum, but at a cost of annihilating most *shear* fluid element deformations. This emerges directly from the approximation of rigid bodies [23], and can be observed in more detail in Figure 6.

3.3.3. Affine Approximations. We account for this lack of shear using the same techniques - we further augment our particles to handle translational modes, by approximating particle velocities as locally *affine* [35]. In the previously constructed

language, we replace $\mathbf{R}_p$ with a *general* $\mathbb{R}^{2,2}$ matrix, rather than an anti-symmetric one. This gives a new meaning to $\omega_p^* \rightarrow \omega_p^\dagger$ in formula (3.3), representing the local approximation of $\nabla \underline{u}_p$ [22]. Following from equation (3.7), we thus replace

$$(3.8) \quad \sum_p w_p^i \underline{u}_p^{\text{rigid}} \rightarrow \sum_p w_p^i \underline{u}_p^{\text{affine}}, \quad \text{where} \quad \underline{u}_p^{\text{affine}} = \underline{u}_p + \omega_p^\dagger (\underline{\chi}_i - \underline{x}_p).$$

Replacing anti-symmetric operators in (3.6) with generalised matrices gives $\omega_p \rightarrow \omega_p^\dagger = \mathfrak{L}_p (\mathfrak{J}_p)^{-1}$, where

$$\mathfrak{J}_p = \sum_g w_p^g (\underline{\chi}_g - \underline{x}_p) (\underline{\chi}_g - \underline{x}_p)^T = \begin{bmatrix} (\Delta x_x^{gp})^2 & \Delta x_x^{gp} \Delta x_y^{gp} \\ \Delta x_x^{gp} \Delta x_y^{gp} & (\Delta x_y^{gp})^2 \end{bmatrix},$$

is analogous to the generalised inertia tensor as defined in equation (3.5), and

$$\mathfrak{L}_p = \sum_g w_p^g \underline{u}_g (\underline{\chi}_g - \underline{x}_p^{\text{old}})^T,$$

is akin to the generalised angular momentum loss tensor, generalised from equation (3.2) [22]. As $\underline{u}_g$ are the divergence-free grid velocities, this calculation must proceed the advection of particles (immediately after the BiLerpGP operator; hence the use of $\underline{x}_p^{\text{old}}$), for use in the next simulation step's BiLerpPG operator (during which we calculate $\mathfrak{J}_p$ using the advected particles). One can show that, in the specific case of multilinear interpolation, we have the identity

$$(3.9) \quad \begin{bmatrix} \frac{\partial w_p^i}{\partial x_{p,x}} \\ \frac{\partial w_p^i}{\partial x_{p,y}} \end{bmatrix} = \nabla w_p^i = w_p^i \mathfrak{J}_p^{-1} (\underline{\chi}_i - \underline{x}_p^{\text{old}}),$$

where we label the components of $\begin{bmatrix} \underline{x}_{p,x} \\ \underline{x}_{p,y} \end{bmatrix} \equiv \underline{x}_p^{\text{old}}$ [22]. Calculating these partial derivatives using the chain rule on the linear interpolation operators, we can use equation (3.9) re-define the interpolation operator (3.8) as

$$(3.10) \quad \sum_p w_p^i \underline{u}_p^{\text{affine}} = \sum_p (w_p^i \underline{u}_p + \mathfrak{L}_p \nabla w_p^i),$$

meaning we never have to invert $\mathfrak{J}_p$ directly - we merely calculate the partial derivatives of w_p^i alongside BiLerpGP, for use in the next simulation step's BiLerpPG.

Finally, we must re-define the concept of total angular momentum in equations (3.1), for our affine particles. Recall that any matrix can be decomposed into a symmetric and anti-symmetric matrix, and that (considering operator (3.3)), anti-symmetric matrices represent purely rotational transformations. Hence, the ‘internal spin’ of a velocity gradient $\nabla \underline{u}_p \approx \mathfrak{L}_p \mathfrak{J}_p^{-1}$ must be extracted from its anti-symmetric component. For APIC, the conservation of total angular momentum is elegantly tracked by said components of the affine matrix $\mathfrak{L}_p$ itself [22], and we therefore write this as a sum of the particles’ ‘orbital’ angular momentum, and their internal spin:

$$(3.11) \quad L_P^{\text{affine}} = \sum_p (\underline{x}_p \times \underline{u}_p) + (\mathfrak{L}_{21} - \mathfrak{L}_{12}),$$

We thus observe APIC in practice. We follow the proofs of C. Jiang [24] that these schemes preserve affine velocity fields, and linear & angular momentum.

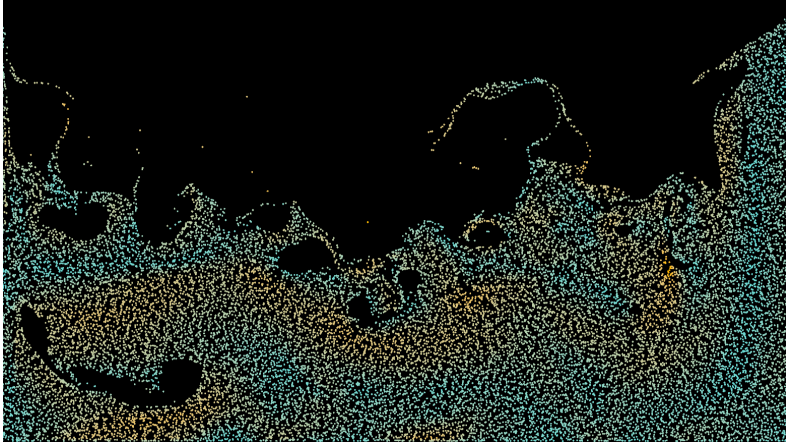


Figure 5: A real-time APIC simulation screenshot, with 25,000 particles. We see that, even for settings involving rough, high forces, many overturning waves, and extreme turbulence, APIC remains stable, and is very effective at preserving vortex motion, even on small scales (such as the small, numerous overturning waves shown in this image).

4. Results.

4.1. Forces on Fluid Elements. Using the APIC angular momentum definitions in formula (3.11), we track the conservation properties of the various models we have derived thus far. First, however, we need confidence that are a suitable approximation for a general fluid flow - in particular, we wish to see emergence of vortices in the limit of high Reynolds number flow, and thus investigate the *shearing* of fluid elements, to see how malleable (and hence inertially-dominated) they are.

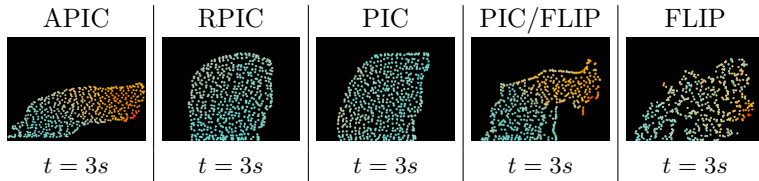


Figure 6: A shear test on a small fluid element, tested with various methods (using a blending factor $\alpha = 0.05$ for PIC/FLIP). 500 fixed particles are initialised in a rectangular shape of height 2.5 times its size (4 particles per cell, distributed uniformly), with no gravity. Each particle is then positionally perturbed in each axis by the factor $2\Phi(s/6) - 1$ (where $\Phi(x)$ denotes the standard normal CDF, and s is the cell size), in order to eliminate periodic visual artefacts. At $t = 0$, a horizontal force $\frac{b}{4\Delta t}$ is applied to each non-solid cell of index (a, b) , giving a linear shear effect. Surface tension is also added to conserve the element's shape, using the divergence controller gain $\kappa = 1.0$.

Unlike PIC and RPIC, which are impervious to shearing, PIC/FLIP, FLIP, and APIC deform as expected. We also remark that $\alpha > 0$ is necessary for fluid elements to overturn, but that *any* presence of PIC prevents the bottom half from shearing effectively. This can be eliminated by using pure FLIP ($\alpha = 1$), at the cost of instability (particularly at the boundary of fluid elements). APIC combines the advantages

of both regimes, with discernable, linear shearing that cleanly overturns the entire length when the structure must rotate.

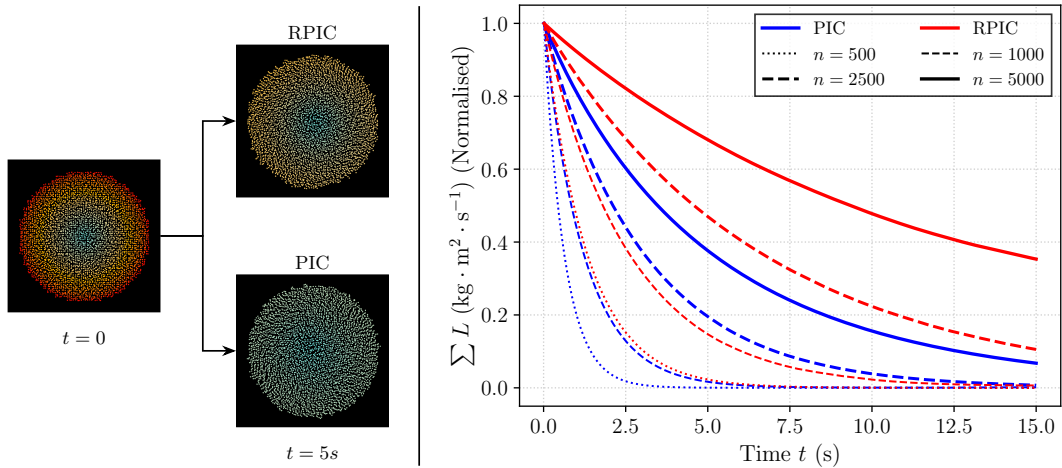


Figure 7: A comparison of the angular momentum conservation of RPIC and PIC, for an oscillating circle of n particles that are initially perturbed with some tangential velocity (with magnitude proportional to the particle’s radius). We choose these models in particular due to a lack of shearing, where energy modes are almost exclusively angular-based - see Figure 6. Sub-cell initial particle positions are as in Figure 6.

We observe that RPIC preserves angular momentum significantly more than PIC; whilst both see more accurate results as n increases (presumably as most angular momentum is stored near the circle’s edges), the *rate* of increase is much faster for RPIC than PIC. RPIC ($n = 5000$) test has the most linear-like decay in the suite.

4.2. Taylor-Green Vortex: Analytical Case Study. The previous examples demonstrate that APIC outperforms the standard PIC model by a considerable margin, but have thus far failed to answer how applicable these methods are for large-scale, inviscid flow problems. To quantitatively benchmark dissipation against an analytical ground truth, therefore, we investigate the developed methods’ capabilities under the canonical 2D ‘Taylor-Green’ vortex problem. Constructed in 1937 [34], the exact solution for this model (in Cartesian coordinates) can be summarised by,

$$(4.1) \quad \underline{u}(x, y, t) = \begin{pmatrix} \sin(\kappa_x x) \cos(\kappa_y y) \\ -\cos(\kappa_x x) \sin(\kappa_y y) \end{pmatrix} e^{-2\nu|\underline{\kappa}|^2 t},$$

a separable equation into an initial vortical perturbation (with $\omega = 2 \sin(x) \sin(y)$ [10]) and exponential damping. We thus require *periodic* boundary conditions, and as such $\kappa_x = \kappa_y = \frac{2\pi}{L}$ denotes the wavenumber for a square simulation of length L [12]. Transitioning away from Dirichlet boundary conditions requires a reconstruction of formulae (2.6) and (2.13), such that cells wrap around the simulation’s edges. We choose the fundamental mode $\kappa = 1$, which produces the vortices depicted in Figure 8. To compare our regimes against the Taylor-Green vortex, we track the total particle kinetic energy, with initial condition given by expression (4.1), which follows,

$$(4.2) \quad E_k = E_0 e^{-4\nu\kappa^2 t}.$$

416 Fitting to various simulations of the Taylor-Green vortex, we recover values of the
 417 kinematic viscosity ν for various APIC simulations, which are presented in Figure 8.

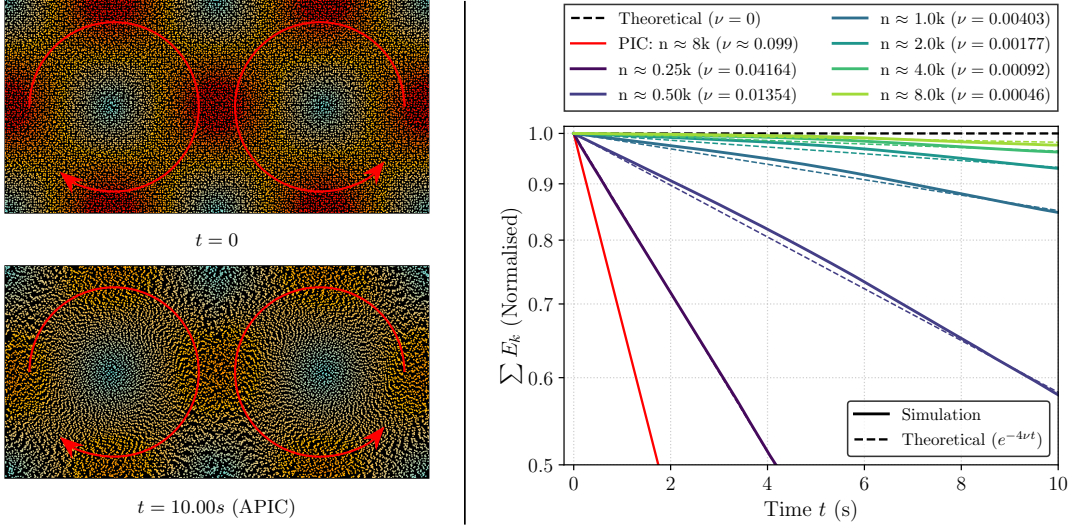


Figure 8: An analysis of the damping properties of APIC, for Taylor-Green vortices. The left-most images show a top-half cross-section of said vortices for a simulation of $32k$ particles, at various times, whilst the right-most figure plots normalised energy over time, for an increasing number of particles per vortex, n (recall that, via the $\kappa = 1$ mode, each simulation contains 4 such vortices). Particles fully populate the grid space, with placements within cells as in Figure 6, and are initially perturbed with velocities $\underline{u}(x, y, 0)$ from equation (4.1). Simulated results are fitted to the theoretical trajectory given by the full solution (4.1), with free parameter ν . We also provide the analytical result for a truly inviscid flow, along with PIC performance, for metrics of comparison. Each test is repeated 5 times for consistency. Crucially, we observe that APIC converges nicely towards the inviscid solution, with a good ‘performance to accuracy’ balance emerging at $n \approx 3k$, after which convergence slows.

418 We make the following remarks based on the above figure. To begin with, despite
 419 being ran by a simulation with $32k$ particles, PIC struggles to conserve energy in
 420 these vortices, and performs on-par with $\approx 0.6k$ APIC particles ($n \approx 0.15k$), therefore
 421 strengthening our confidence in the results obtained in Figure 7. Further, for many
 422 APIC results we observe a characteristic ‘settling’ time, before which there is a slight
 423 upward deviation from the expected exponential decay of energy. This is a well-
 424 documented phenomenon [16] and, as this behaviour does not arise with the FLIP
 425 scheme, it is unlikely that we have probed the instability of APIC. Rather, this is likely
 426 due to incorrect initialisation of $\mathfrak{L}_p$ (this is set theoretically using $\nabla \underline{u}$ of formula (4.1),
 427 and does not take into account numerical errors in interpolation and projection).
 428 However, further analysis beyond this work is required to better characterise this
 429 behaviour.

430 Further numerical experiments were conducted with $n \approx 16k$ APIC particles,
 431 which recovered a theoretical value of $\nu = 3.7 \times 10^{-4}$ - a modest improvement over the
 432 best results seen so far. Making use of non-dimensional parameters (using $L = 98.4u$

to fit all our particles, where u is the standard Unity Game Engine unit), we derive an APIC Reynolds number of $Re = 5420$, whilst PIC yields $Re = 40.51$ for the same number of particles. These are upper bounds, with more realistic results (obtained using $n \approx 4k$) being $Re = 2213.68$ for APIC, and $Re = 10.27$ for PIC respectively. From this, we can conclude that *only* APIC (even at lower particle counts) has the ambition of being used for large-scale applications that model inviscid flows, and that, upon moving to higher particle counts, we appear to converge arbitrarily close to the inviscid solution (4.1). It is also worth mentioning that FLIP exhibits negligible kinetic energy decay in the test, due to its inherent energy conservation properties: for $n \approx 8k$, we yield a value of $\nu = 1.26 \times 10^{-4}$, corresponding to a Reynolds number approximately four times greater than that of APIC. Without an established wider benchmark, due to a distinct lack of research in this area, one cannot be sure whether these values are reproducible. Nonetheless, this serves as a novel validation for these constructed regimes, giving a metric of comparison with other CFD methods.

5. Conclusion. In this paper, we have constructed a hybrid Particle-In-Cell method, that offers a synergistic approach of Lagrangian advection and Eulerian projection techniques. We enforce this divergence-free property through the (Pre-conditioned) CG Method, which required bijecting our MAC grid representation of the velocity field $\underline{u}$ to a sparse linear equation, consisting of the fluid’s divergence and information of each cell’s neighbours. The largest focus of our investigation was spent on the main crux of these hybrid methods: the velocity transfers between the Lagrangian and Eulerian regimes. After building up the knowledge of how these interpolation transfer operators work, we carefully considered their inherent problems (namely, the loss of particle degrees of freedom), and the various entry points into solving these issues. The most promising of these ideas, that being to augment our particles with scalars or matrices representing angular momentum information, formed the premise of RPIC and APIC respectively. This required careful consideration to quantify the angular momentum and inertia of particle states, but rewards us with *significantly* stronger angular momentum conservation properties, compared to that of PIC. Whilst RPIC (like PIC) is ill-suited for shearing (despite having the sufficient conservation properties), augmenting with a full angular momentum matrix (rather than the anti-symmetric components in RPIC) restores these shearing modes, giving us the APIC method. This shows remarkable potential in terms of visual quality, local volume conservation, and the physical accuracy of turbulence (both in respect to PIC and other industry standards).

As computation of the velocity gradient matrices in APIC, and addition to the CG Method solver, does not add significant performance overhead to the simulation ($\lesssim 10.1\%$ in testing), and is incredibly well suited for parallelisation (due to our assumptions of particle momentum depending only on their *fixed* grid node neighbours, rather than their particle neighbours), whilst also retaining the stability of PIC over methods such as FLIP, it can be considered a definitive upgrade over standard PIC.

Pure grid-based methods (such as *Stable Fluids* [33]), while applicable for smooth, flow fluid motion, struggle to preserve angular information due to the sharp edges of cells, and apparent ‘blurriness’ of the pressure field over time [8]. Meanwhile, Lagrangian-based methods (such as SPH: *Smooth Particle Hydrodynamics* [28]) are incredibly unstable, and suffer from long-term volume leaking and particle clumping, making them a questionable solution to the Navier-Stokes equation [19]. These methods require the addition of external viscous forces (which themselves do not conserve angular momentum [20]) in order to well-approximate the low viscosity of

fluids such as water. Quantifying the vorticity of a rigid body is also significantly easier in PIC methods (in Section 3.3.2, χ is simply the *MAC grid* components encompassing p) than in SPH, which requires an expensive local smoothing kernel and nearest-neighbour search about p [20, 21].

Despite the advantages, APIC (which, like PIC, is still based on the assumption of material continuity) suffers too from the problem of particle clumping, and therefore cannot effectively handle particle separation [36]. Resultantly, over time particles will begin to merge together and behave ‘as one’, reducing the degrees of freedom of the particle representation [27]. Some techniques are available to account for this, however (albeit at a performance cost), such as adding an intrinsic repulsion force to pairs of particles, whenever they cross some distance threshold. Implementing this effectively requires the use of the Eulerian grid to check only the particles in some neighbourhood.

Nonetheless, this paper successfully provides the sufficient groundwork for extending these methods to new and exciting domains. For instance, by revisiting the assumption of treating air as a vacuum in Section 2.2.1, there are several considerations for future work, such as modelling wave generation by surface winds, or persistent bubbles inside our fluid [27] (both of which make use of the presence and compressibility of air). Further, one can consider rigorous implementation of surface tension in our fluid: in our transition from PIC/FLIP to APIC, we appear to lose some inherent clumping of particles near the boundary of fluid elements. As illustrated through Figure 6, this presumably stems from reducing numerical instability near boundaries. Hence, it is perhaps more natural to use FLIP when introducing surface tension effects; indeed, this is the case for many new schemes, such as Yi-Lun Chen [9].

Despite these (albeit surmountable) limitations, it is clear that Affine Particle-In-Cell methods will continue to develop an incredibly promising legacy in the world of large-scale Computational Fluid Dynamics. Having only been developed recently in 2015 [22], future work can easily expand the outlined formulations to many new domains and regimes, and one can therefore be positive that new and exciting research will be undoubtedly sparked by the robust solutions that APIC provides.

Code Availability. The source code and parameters used to generate the simulations, figures, and statistics presented in this paper are available on GitHub.

REFERENCES

- [1] *Vortex Bladeless: A Wind Generator without Blades*. <https://vortexbladeless.com>, 2026.
- [2] D. ANIMATION, *Moana: Crashing waves*. https://media.disneyanimation.com/uploads/production/publication_asset/164/asset/Moana-Crashing-Waves.pdf, 2016.
- [3] S. AXLER, *Linear Algebra Done Right*, Springer, 3rd ed., 2015.
- [4] C. S. BAIRD, *Helmholtz decomposition of vector fields*. University of Massachusetts Lowell, 2013, https://faculty.uml.edu/cbaIRD/all_notes/re_helmholtz.pdf.
- [5] J. BRACKBILL, D. KOTHE, AND H. RUPPEL, *Flip: A low-dissipation, particle-in-cell method for fluid flow*, Computer Physics Communications, 48 (1988), pp. 25–38, [https://doi.org/https://doi.org/10.1016/0010-4655\(88\)90020-3](https://doi.org/https://doi.org/10.1016/0010-4655(88)90020-3), <https://www.sciencedirect.com/science/article/pii/0010465588900203>.
- [6] J. U. BRACKBILL AND H. M. RUPPEL, *Flip: A method for adaptively zoned, particle-in-cell calculations of fluid flows in two dimensions*, Journal of Computational physics, 65 (1986), pp. 314–343.
- [7] C. BRALEY AND A. SANDU, *Fluid simulation for computer graphics: A tutorial in grid based and particle based methods*, Virginia Tech, Blacksburg, 3 (2010).
- [8] R. BRIDSON, *Fluid Simulation for Computer Graphics*, Taylor & Francis, 2008, <https://books>.

- google.co.uk/books?id=gFI8y87VCZ8C.
- [9] Y.-L. CHEN, J. MEIER, B. SOLENTHALER, AND V. C. AZEVEDO, *An extended cut-cell method for sub-grid liquids tracking with surface tension*, ACM Transactions on Graphics (TOG), 39 (2020), pp. 1–13.
 - [10] Y. DAGAN, *Analytical solutions for particle dispersion in taylor–green vortex flows: Y. dagan*, Theoretical and Computational Fluid Dynamics, 39 (2025), p. 14.
 - [11] O. DING, T. SHINAR, AND C. SCHROEDER, *Affine particle in cell method for mac grids and fluid simulation*, Journal of Computational Physics, 408 (2020), p. 109311.
 - [12] D. DRIKAKIS, C. FUREBY, F. F. GRINSTEIN, AND D. YOUNGS, *Simulation of transition and turbulence decay in the taylor–green vortex*, Journal of Turbulence, (2007), p. N20.
 - [13] D. ENGLESSON, *Real-time FLIP fluid simulation on the GPU*, master’s thesis, Lund University, 2016.
 - [14] M. W. EVANS, F. H. HARLOW, AND E. BROMBERG, *The particle-in-cell method for hydrodynamic calculations*, tech. report, 1957.
 - [15] G. L. EYINK, *Josephson-anderson relation and the classical d’alembert paradox*, Physical Review X, 11 (2021), p. 031054.
 - [16] B. D. FUDGE, R. CIMPEANU, AND A. A. CASTREJÓN-PITA, *Drop impact onto immiscible liquid films floating on pools*, Scientific Reports, 14 (2024), p. 13671.
 - [17] H. GOLDSTEIN, C. POOLE, AND J. SAFKO, *Classical Mechanics*, Addison Wesley, 3rd ed., 2002.
 - [18] G. H. GOLUB AND C. F. VAN LOAN, *Matrix Computations*, Johns Hopkins University Press, 4th ed., 2013.
 - [19] Y. HE, H. LI, Y. CHEN, AND X. CHEN, *Incompressible fluid simulation: A comparison*, tech. report, University of Southern California, 2025.
 - [20] X. HU AND N. ADAMS, *Angular-momentum conservative smoothed particle dynamics for incompressible viscous flows*, Physics of Fluids, 18 (2006).
 - [21] M. IHMSEN, N. AKINCI, M. BECKER, AND M. TESCHNER, *A parallel sph implementation on multi-core cpus*, in Computer Graphics Forum, vol. 30, Wiley Online Library, 2011, pp. 99–112.
 - [22] C. JIANG, C. SCHROEDER, A. SELLE, J. TERAN, AND A. STOMAKHIN, *The affine particle-in-cell method*, ACM Trans. Graph., 34 (2015), <https://doi.org/10.1145/2766996>, <https://doi.org/10.1145/2766996>.
 - [23] C. JIANG, C. SCHROEDER, A. SELLE, J. TERAN, AND A. STOMAKHIN, *The affine particle-in-cell method (video supplement)*. <https://www.youtube.com/watch?v=jPG5H5ZoL5Y>, 2015.
 - [24] C. JIANG, C. SCHROEDER, A. SELLE, J. TERAN, AND A. STOMAKHIN, *Tech report: the affine particle-in-cell method*, (2025).
 - [25] C.-J. LIN AND J. J. MORÉ, *Incomplete cholesky factorizations with limited memory*, SIAM Journal on Scientific computing, 21 (1999), pp. 24–45.
 - [26] W. MCCROSKEY AND J. PHILIPPE, *Unsteady viscous flow on oscillating airfoils*, AIAA journal, 13 (1975), pp. 71–79.
 - [27] M. MÜLLER, *FLIP fluid simulation*. <https://matthias-research.github.io/pages/tenMinutePhysics/18-flip.pdf>, 2021.
 - [28] M. MÜLLER, D. CHARYPAR, AND M. GROSS, *Particle-based fluid simulation for interactive applications*, in Proceedings of the 2003 ACM SIGGRAPH/Eurographics symposium on Computer animation, 2003, pp. 154–159.
 - [29] Z. QU, M. LI, F. DE GOES, AND C. JIANG, *The power particle-in-cell method*, ACM Transactions on Graphics, 41 (2022).
 - [30] F. SCHULZE, *MA263 - multivariable analysis*. Lecture Notes, University of Warwick, 2025.
 - [31] S. SHAHRABI, *Gentle introduction to fluid simulation for programmers and technical artists*. Medium, 2017, <https://shahriyarsahrabi.medium.com/gentle-introduction-to-fluid-simulation-for-programmers-and-technical-artists-7c0045c40bac>.
 - [32] J. R. SHEWCHUK, *An introduction to the conjugate gradient method without the agonizing pain*, tech. report, Carnegie Mellon University, 1994.
 - [33] J. STAM, *Stable fluids*, in Proceedings of the 26th annual conference on Computer graphics and interactive techniques (SIGGRAPH), 1999, pp. 121–128.
 - [34] G. I. TAYLOR AND A. E. GREEN, *Mechanism of the production of small eddies from large ones*, Proceedings of the Royal Society of London. Series A-Mathematical and Physical Sciences, 158 (1937), pp. 499–521.
 - [35] M. TUPEK, J. KOESTER, AND M. MOSBY, *A momentum preserving frictional contact algorithm based on affine particle-in-cell grid transfers*, arXiv preprint arXiv:2108.02259, (2021).
 - [36] C. WANG, J. ZHANG, C. LI, AND C. WANG, *Dc-apic: A decomposed compatible affine particle in cell transfer scheme for non-sticky solid–fluid interactions in mpm*, Graphical Models, 139 (2025), p. 101269.